Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192521157
Student Name	K. KABILAN

Assumptions used throughout. n = number of products (or requests for M2); servers execute 10^8 basic operations per second, so the 10-minute (600 s) nightly window allows at most 6×10^{10} operations. Where a recurrence is given without a base value, T(1) = 1 (a constant) is assumed; this affects only lower-order terms, never the asymptotic result. log means $\log_2$.

(a) Understand the Problem

Module	Input size	Basic operation	Where it occurs	Growth of its count with n
M1 Duplicate detector	n = number of products	Comparison Product[i].title = Product[j].title (and seller test)	Condition of the IF in the inner FOR loop	Executes once for every pair (i, j), i < j: n(n−1)/2 times → quadratic
M2 Rate limiter	n = number of requests	Audit(Request[j])	Body of the inner FOR loop (marked "basic operation")	Inner loop runs i = 1, 2, 4, … times → geometric sum ≈ n, linear
M3 – S1 Sequential search	n = size of catalogue	Key comparison with Product[k]	Once per iteration of the scan	Up to n comparisons → linear
M3 – S2 Binary search	n = size of sorted index	Key comparison with middle entry	Once per halving step (T(n) = T(n/2) + 1)	Increases by 1 each time n doubles → log n
M4 Recommendation engine	n = customer–product data	Combining (merge) of one partial-result element	Merge step costing n per call, outside the recursive calls	Recursion gives n^2 (4 sub-problems) or n log n (2 sub-problems)
M5 Sales analytics	n = number of records	Record comparison in the sort done at every level	The n log n term of each recursive call; report generator: its f(n) operations	Engine ≈ n $\log^2$ n; report generator f(n) = $3n^2$ + … → quadratic

Justification. In each module the chosen operation sits in the innermost loop or in the non-recursive part of the recursion, so all other work (index updates, loop control) is a constant multiple of it. Its count therefore captures the growth of the running time and is independent of machine and language.

(b) Analyse the Iterative Modules

M1 – exact comparisons, time and space

For a fixed i (0 ≤ i ≤ n − 2) the inner loop runs j = i + 1 … n − 1, that is (n − 1 − i) times. Hence

$$C_1(n) = \Sigma\ (i = 0 \text{ to } n-2)\ (n - 1 - i) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2$$

The count is the same for every input (the loops do not depend on the data), so best = average = worst case. **$C_1(n) = n(n-1)/2 = \frac{1}{2}n^2 - \frac{1}{2}n \in \Theta(n^2)$.** For n = 100,000 this is 4,999,950,000 ≈ 5×10^9; for n = 1,000,000 it is 499,999,500,000 ≈ 5×10^{11}.

Time complexity: $\Theta(n^2)$. Space complexity: $\Theta(1)$ extra space (only i, j and the flag set by MarkDuplicate; the product array itself is the input).

M2 – exact number of Audit operations

The outer loop takes i = 1, 2, 4, …, 2^k while i < n; the value i is also the number of times the inner loop runs (j = 0 … i − 1). If the outer loop runs m times, then i = 2^0, 2^1, …, 2^(m−1) and m = $\lceil \log_2 n \rceil$ (the number of powers of 2 smaller than n). The number of Audit operations is

$$A(n) = \Sigma\ (k = 0 \text{ to } m-1)\ 2^k = 2^m - 1 = 2^{\lceil \log_2 n \rceil} - 1$$

Since 2^(m−1) < n ≤ 2^m, we get **$n - 1 \le A(n) < 2n$**, so $A(n) \in \Theta(n)$.

Verdict on the claim: the team member's $\Theta(n \log n)$ is incorrect. The reasoning "outer loop log n times × inner loop n times" overestimates, because the inner loop runs only i times, not n times, and i doubles. The sum is a geometric series dominated by its last term. ($O(n \log n)$ is a true but loose upper bound; it is not tight.) **Tight bound: $\Theta(n)$**; time $\Theta(n)$, space $\Theta(1)$.

Verification for n = 1,024: the outer loop runs for i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 iterations, since 1024 < 1024 is false). Audits = 1 + 2 + 4 + … + 512 = 1,023 = $2^{10} - 1 = n - 1$. The claimed n log n would give 1,024 × 10 = 10,240, about ten times too large.

(c) Solve a Recurrence by Substitution (Search version S2)

Recurrence: T(n) = T(n/2) + 1, T(1) = 1. Take $n = 2^k$ and substitute backwards:

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &= [T(n/4) + 1] + 1 = T(n/4) + 2 \\ &= [T(n/8) + 1] + 2 = T(n/8) + 3 \\ &\ldots \\ &= T(n/2^i) + i \qquad \text{(pattern after i substitutions)} \end{aligned}$$

The base case is reached when $n/2^i = 1$, i.e. $i = k = \log_2 n$. Then

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

Closed form: $T(n) = \log_2 n + 1 \in \Theta(\log n)$ (for general n, $\lfloor \log_2 n \rfloor + 1$). Check: T(8) = T(4)+1 = T(2)+2 = T(1)+3 = 4 = 1 + $\log_2 8$. ✓

Implementation	Time complexity	Space complexity
Iterative (loop on low, high)	$\Theta(\log n)$ – at most $\lfloor \log_2 n \rfloor + 1$ iterations	$\Theta(1)$ – only low, high, mid
Recursive	$\Theta(\log n)$ – the same recurrence T(n) = T(n/2) + 1	$\Theta(\log n)$ – call-stack depth $\lfloor \log_2 n \rfloor + 1$ (about 20 frames for n = 10^6)

(d) Apply the Master Theorem

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a).

Design	a	b	f(n)	n^(log_b a)	Case used	Result
M4 current: T(n) = 4T(n/2) + n	4	2	n	n^2	Case 1: f(n) = O(n^(2−ε)) with ε = 1	$\Theta(n^2)$

Design	a	b	f(n)	n^(log_b a)	Case used	Result
M4 redesign: T(n) = 2T(n/2) + n	2	2	n	n	Case 2: f(n) = Θ(n^(log_b a))	Θ(n log n)

When the data doubles: current design: T(2n)/T(n) → 2^2= **4×** (n^2 growth). Redesign: T(2n)/T(n) = 2(1 + 1/$\log_2$ n) ≈ **2.1×** at n = 10^6 (slightly more than 2×). Removing the replicated partitions (4 sub-problems → 2) changes the exponent from 2 to 1, so the redesign scales far better.

M5 – T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, n^(log_b a) = n, f(n) = n log n.

- Case 1 does not apply: f(n) is larger than n^(log_b a), not smaller.
- Case 2 (basic form) needs f(n) = Θ(n); n log n is not Θ(n).
- Case 3 needs f(n) = Ω(n^(1+ε)) for some ε > 0; but log n = o($n^ε$) for every ε > 0, so n log n is not polynomially larger than n. **None of the three standard cases applies** (f lies in the gap between Cases 2 and 3).

Extended Master Theorem: if f(n) = Θ(n^(log_b a) · $\log^k$ n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here k = 1, so **T(n) = Θ(n $\log^2$ n)**.

Recursion-tree check: level i has 2^i sub-problems of size $n/2^i$, each costing $(n/2^i)\cdot\log(n/2^i)$; the level total is n(log n − i). Summing the log n levels:

$$\Sigma_{i=0}^{\log n-1} n(\log n - i) = n(\log n + (\log n - 1) + \dots + 1) = n \cdot \frac{\log n(\log n + 1)}{2} = \Theta(n \log^2 n) \quad \checkmark$$

(e) Analyse Best, Average and Worst Cases

	Best case	Average case	Worst case
S1 – sequential scan	Product is the first entry → 1 comparison	Product present, equally likely at any position → (n+1)/2; with probability p of presence see below	Product is last, or absent → n comparisons
S2 – binary search on sorted index	Product equals the middle entry of the index → 1 comparison	Successful: ≈ $\log_2(n+1) - 1$; unsuccessful: ≈ $\log_2$ n	Product absent or found at the deepest level → $\lfloor \log_2 n \rfloor + 1$ comparisons

S1 – average number of comparisons when the product exists with probability p

If present, it is at position i (1 ≤ i ≤ n) with probability p/n each and costs i comparisons; if absent (probability 1 − p) it costs n:

$$C_{avg}(n) = \Sigma_{i=1}^{n} i \cdot (p/n) + n(1-p) = (p/n) \cdot n(n+1)/2 + n(1-p) = p(n+1)/2 + n(1-p)$$

For n = 1,000,000 and p = 0.5: C_avg = 0.5 × (1,000,001)/2 + 1,000,000 × 0.5 = 250,000.25 + 500,000 = **750,000.25 ≈ 7.5 × 10^5 comparisons.** (Limits: p = 1 gives 500,000.5; p = 0 gives 1,000,000.) For S2 at n = 10^6: best 1, average ≈ 18.9, worst 20 comparisons.

(f) Prove an Asymptotic Bound

Given f(n) = 3n^2 + 10n $\log_2$ n + 50. To prove f(n) ∈ Θ(n^2) we need constants c_1, c_2 > 0 and n_o with $c_1n^2 \le f(n) \le c_2n^2$ for all n ≥ n_o.

Lower bound. For n ≥ 1, $\log_2$ n ≥ 0, so 10n $\log_2$ n + 50 > 0 and f(n) ≥ 3n^2. Take **c_1 = 3**.

Upper bound. For n ≥ 1, $\log_2$ n ≤ n, so 10n $\log_2$ n ≤ 10n^2; also 50 ≤ 50n^2. Therefore f(n) ≤ 3n^2 + 10n^2 + 50n^2 = 63n^2. Take **c_2 = 63, n_o = 1**.

So $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$, hence **$f(n) \in \Theta(n^2)$**. (A tighter pair also works: $c_2 = 4$ with $n_0 = 60$, e.g. $f(60) = 14{,}394 \le 4 \times 3{,}600 = 14{,}400$.)

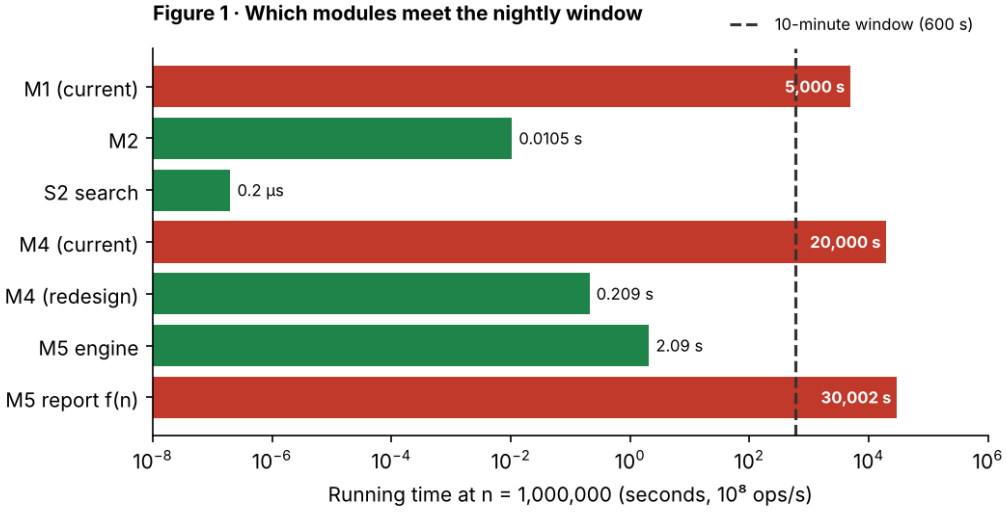
f(n) is not O(n). Suppose $f(n) \le c \cdot n$ for some constant $c > 0$ and all $n \ge n_0$. Since $f(n) \ge 3n^2$, this gives $3n^2 \le c \cdot n$, i.e. $n \le c/3$, for every $n \ge n_0$. This is false for any $n > \max(n_0, c/3)$. The contradiction shows no such c and n_0 exist, so $f(n) \notin O(n)$ (consistently, $f(n) \in \Omega(n^2)$).

(g) Scalability Decision

n = 1,000,000; speed 10^8 operations/s; window 10 min = 600 s (capacity 6×10^{10} operations).

Module	Operations at n = 10^6	Running time	Fits 600 s?	Margin
M1 (current)	$n(n-1)/2 = 4.99999 \times 10^{11}$	≈ 5,000 s ≈ 83.3 min	**No**	8.3× too slow
M2	$2^{20} - 1 = 1{,}048{,}575 \approx 1.05 \times 10^6$	≈ 0.0105 s	Yes	≈ 57,000× spare
S2 (binary search)	$\lfloor \log_2 n \rfloor + 1 = 20$	≈ 0.2 µs per search	Yes	Negligible
M4 current (4T(n/2) + n)	$2n^2 - n \approx 2.0 \times 10^{12}$	≈ 20,000 s ≈ 5.6 h	**No**	33× too slow
M4 redesign (2T(n/2) + n)	$n \log_2 n + n \approx 2.09 \times 10^7$	≈ 0.21 s	Yes	≈ 2,900× spare
M5 analytics engine	$\approx \frac{1}{2} n \log^2 n \approx 2.09 \times 10^8$	≈ 2.1 s	Yes	≈ 290× spare
M5 report generator $f(n) = 3n^2 + 10n \log n + 50$	$\approx 3.0002 \times 10^{12}$	≈ 30,002 s ≈ 8.3 h	**No**	50× too slow

Exact solutions used: $T(n) = 2n^2 - n$ for the current M4 (verify: $4T(n/2) + n = 2n^2 - 2n + n$); $T(n) = n \log n + n$ for the redesign; $n \log n (\log n + 1)/2 + n$ for the M5 engine ($n = 2^k$, $T(1) = 1$).



Recommendation

Modules that will fail the 10-minute window at n = 1,000,000: M1, the current M4 design and the M5 report generator. All three are $\Theta(n^2)$, so growing the catalogue ten-fold multiplies their cost by 100 (M1 already needs ≈ 50 s at n = 100,000 and becomes 5,000 s at 10^6).

- **M1 – redesign:** replace the all-pairs comparison by hashing on the key (title, seller): insert each product into a hash set and mark it as a duplicate when its key is already present. Expected $\Theta(n)$ ≈

10^6 operations (≈ 0.01 s). Alternative: sort by (title, seller) in $\Theta(n \log n) \approx 2 \times 10^7$ operations (≈ 0.2 s) and compare neighbours only.

- **M4 – adopt the proposed redesign:** removing the replicated partitions changes $T(n) = 4T(n/2) + n$ ($\Theta(n^2)$) into $T(n) = 2T(n/2) + n$ ($\Theta(n \log n)$), reducing 5.6 h to ≈ 0.2 s.
- **M5 report generator – redesign:** the $\Theta(n^2)$ term ($3n^2$) must be replaced by an $O(n \log n)$ method, for example sorting/aggregating the records once or using an index instead of nested scans. The remaining $10n \log_2 n \approx 2 \times 10^8$ operations would run in about 2 s.
- **Retain:** M2 (linear, about 10^6 operations), S2 binary search (20 comparisons; keep the product index sorted, a one-off $\Theta(n \log n)$ cost, in preference to S1 which needs up to 10^6 comparisons per search) and the M5 analytics engine ($\Theta(n \log^2 n)$, ≈ 2 s).

Summary of Results

Module / item	Result	Interpretation
M1	$n(n-1)/2 \in \Theta(n^2)$; space $\Theta(1)$	Fails at $n = 10^6$; redesign
M2	$2^{\lceil \log_2 n \rceil} - 1 \in \Theta(n)$; $n = 1{,}024 \rightarrow 1{,}023$	$\Theta(n \log n)$ claim is wrong
S2	$T(n) = \log_2 n + 1 \in \Theta(\log n)$	Iterative space $\Theta(1)$, recursive $\Theta(\log n)$
M4	$4T(n/2)+n = \Theta(n^2)$; $2T(n/2)+n = \Theta(n \log n)$	Use the redesign
M5	$2T(n/2) + n \log n = \Theta(n \log^2 n)$	No standard case; extended theorem
S1 average	$p(n+1)/2 + n(1 - p) = 750{,}000.25$	$n = 10^6$, $p = 0.5$
f(n)	$3n^2 \le f(n) \le 63n^2$ for $n \ge 1$; $f \notin O(n)$	$\Theta(n^2)$; report generator must be redesigned

Code of Conduct

I **K. KABILAN / 192521157** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Kabilan Date: 28-10-2026